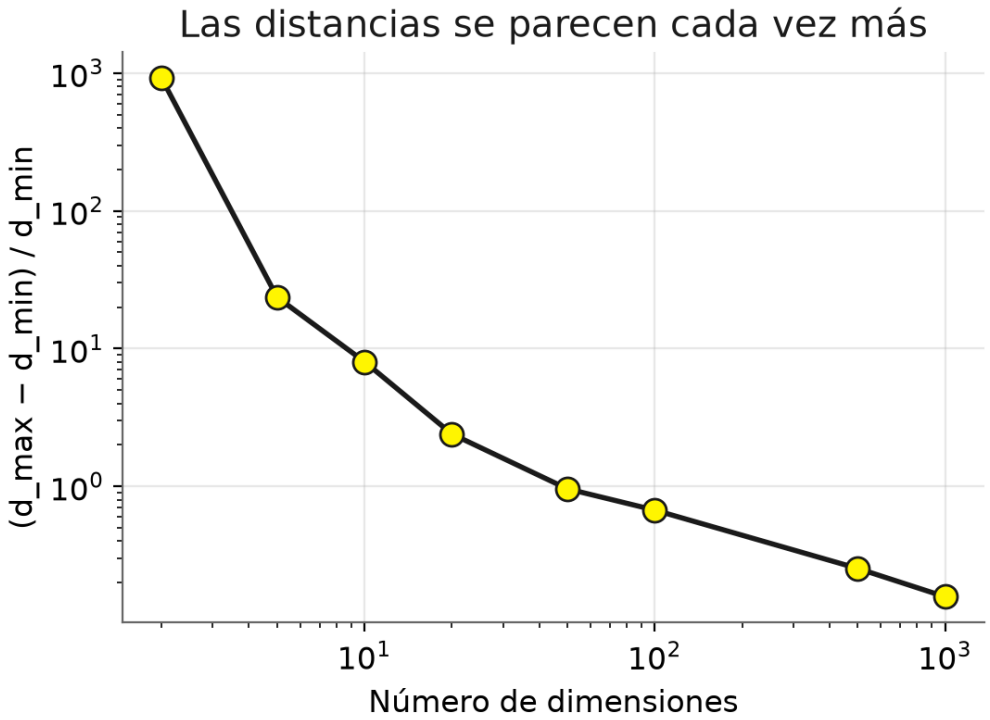
Análisis de Componentes Principales (PCA)

Aprendizaje No Supervisado

UDEM Jorge Alberto Ramírez García, M.Eng.

La maldición de la dimensionalidad

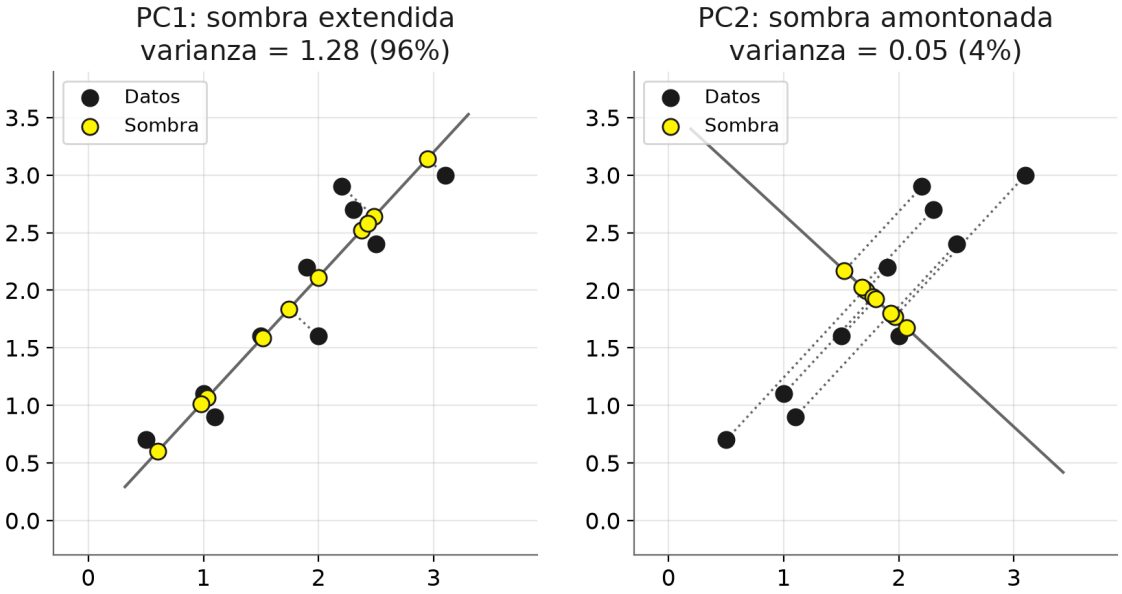


Con muchas columnas…

- Las distancias entre puntos **se parecen**: "cerca" y "lejos" pierden sentido
- K-Means y DBSCAN dependen de distancias
- Muchas variables son **redundantes** (dicen lo mismo)
- No podemos **ver** más de 3 dimensiones

Idea: describir los datos con **menos columnas** que conserven casi toda la información.

Intuición: la sombra que conserva más



La sombra

- Proyectar = mirar la sombra de los datos
- PCA elige la dirección donde la sombra queda **más extendida**
- Más varianza = más información

Maximizar la varianza de la sombra = minimizar las distancias punteadas (lo que se pierde).

El algoritmo en 4 pasos

1. **Estandarizar**
   Media 0 y desviación 1 en cada variable (StandardScaler)

2. **Matriz de covarianza**
   $C = (1 / (n - 1)) \cdot X^T X$: cómo varían las variables juntas

3. **Eigenvectores y eigenvalores**
   $C \cdot v = \lambda \cdot v \rightarrow$ v = dirección, λ = varianza; ordenar de mayor a menor λ

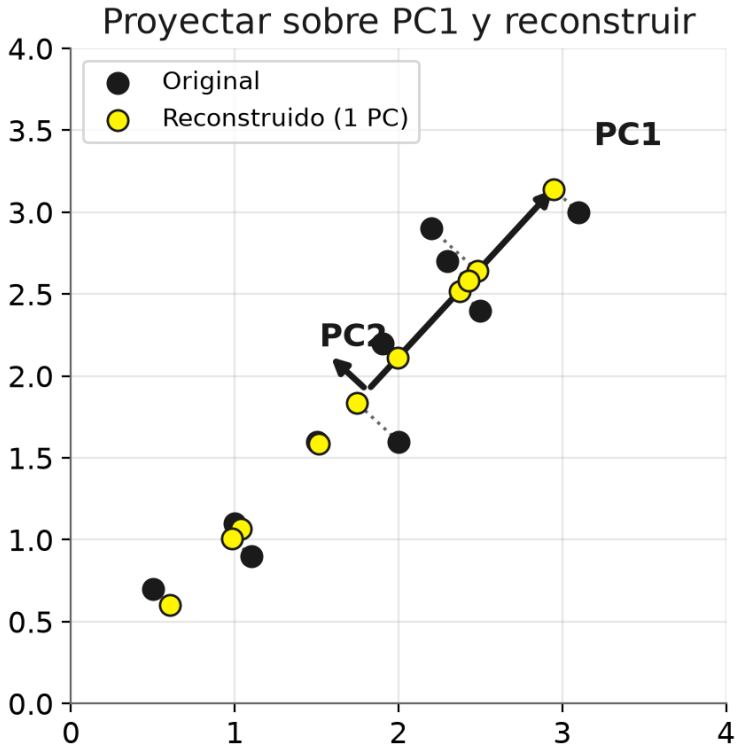
4. **Proyectar**
   $Z = X \cdot V_k$: coordenadas de cada punto en los k primeros componentes

Detalles que importan

- C es simétrica → **np.linalg.eigh**
- eigh regresa λ de **menor a mayor**: hay que ordenar
- Los componentes son **perpendiculares** entre sí
- El signo de cada v es arbitrario

Varianza explicada por $PC_i = \lambda_i / \Sigma \lambda_j$

Proyectar y reconstruir



- **Proyectar**: cada punto pasa de 2 números a 1 (su posición sobre PC1)
- **Reconstruir**: regresar al espacio original → $\hat{x} = z \cdot v_1^T + \mu$
- Los puntos reconstruidos caen sobre la recta de PC1
- La distancia punteada es el **error de reconstrucción**

Lo que se pierde al reconstruir es exactamente la varianza de los componentes que descartamos (los λ que no usamos).

¿Por qué estandarizar antes de PCA?

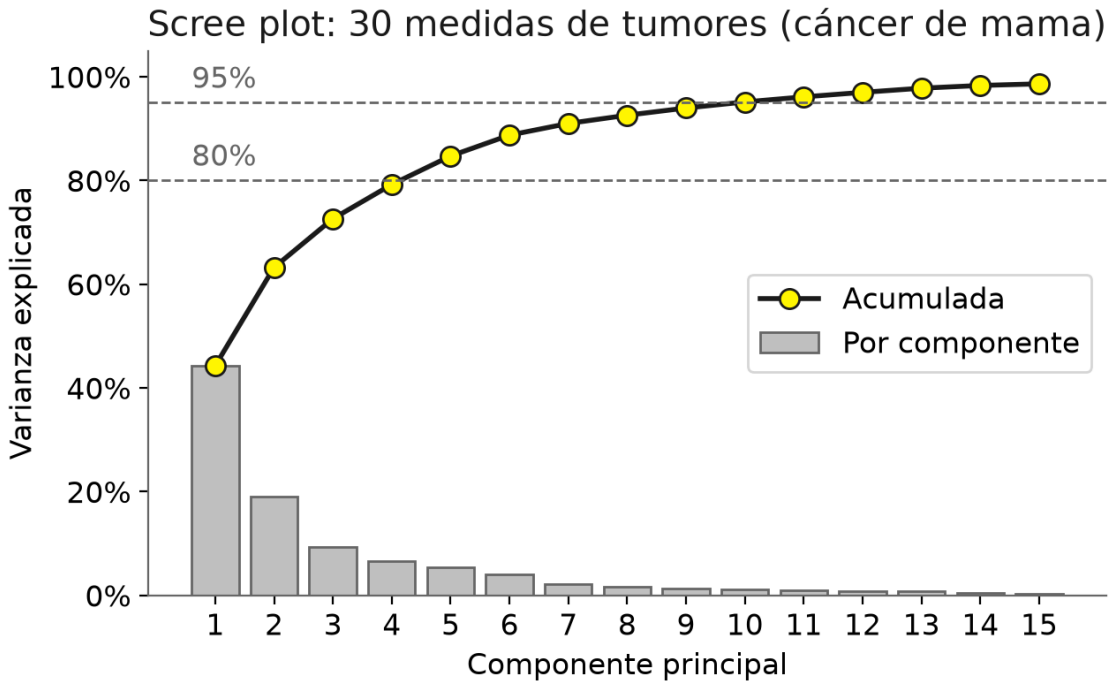
200 personas: altura y peso muy relacionados (r = 0.78); ingreso independiente.

	Desv. estándar	PC1 sin escalar	PC1 escalado
altura_cm	8.8	0.00	0.71
peso_kg	9.7	0.00	0.71
ingreso_mensual	8,132	1.00	−0.06
Varianza explicada por PC1		100%	59%

Sin escalar: PC1 = la variable con los números más grandes. No es un patrón, es un problema de unidades.

Escalado: PC1 = "tamaño corporal" (altura + peso). Aparece la estructura real.

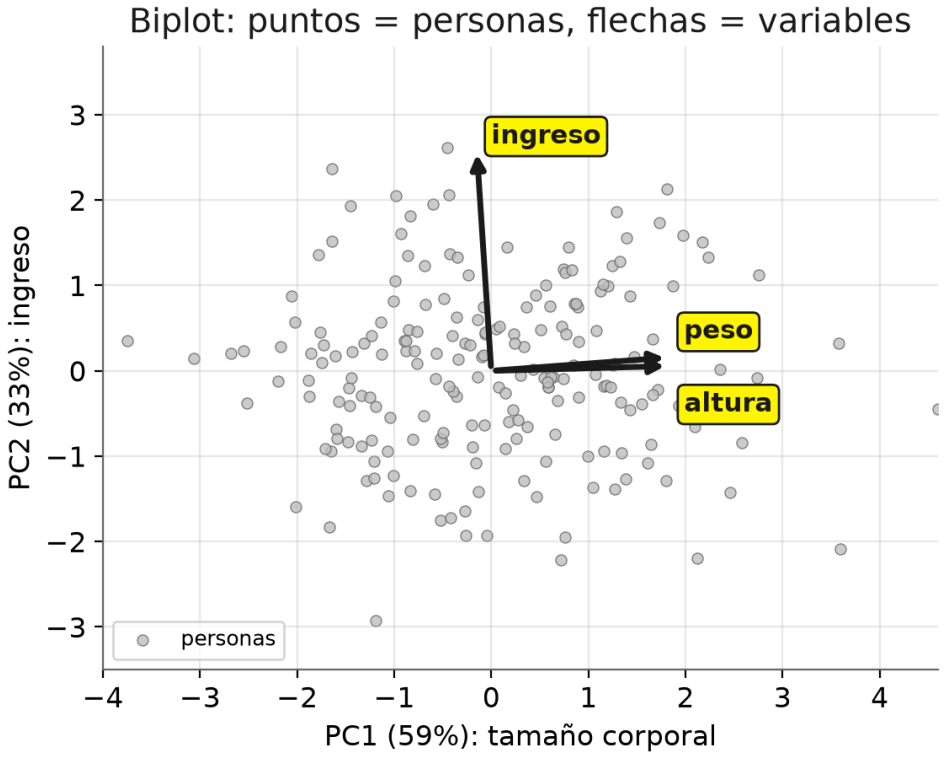
Varianza explicada y scree plot



¿Cuántos componentes?

- **Varianza acumulada**: 80% a 95% (compresión)
- **Codo**: donde las barras dejan de caer
- **Kaiser**: $\lambda > 1$ con datos estandarizados
- **Visualizar**: 2 o 3
- Los criterios pueden no coincidir: decide por el **objetivo**

Cargas y biplot



Cargas (loadings)

- Peso de cada variable en el componente (pca.components_)
- Mismo signo = suben juntas
- Un PC es una **mezcla**: ponle un nombre de negocio

Biplot

- Flechas juntas = variables correlacionadas (altura y peso)

¿Cuándo usar PCA?

Visualización

- Ver datos de muchas variables en 2D o 3D
- Descubrir grupos y atípicos
- Biplot para explicar

Compresión

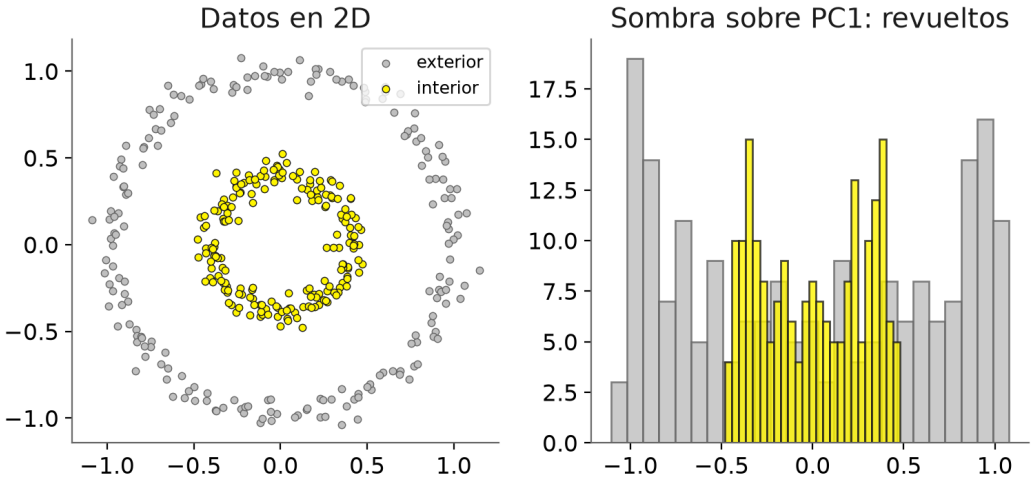
- Guardar menos números por registro
- Imágenes: 784 píxeles → ~150 componentes (95%)
- Quitar ruido de las últimas direcciones

Preprocesamiento

- Antes de K-Means o DBSCAN con muchas variables
- Elimina redundancia y acelera
- Compara silhouettes en el **mismo espacio** de referencia

PCA reduce columnas, no mediciones: cada componente usa todas las variables originales.

Limitaciones



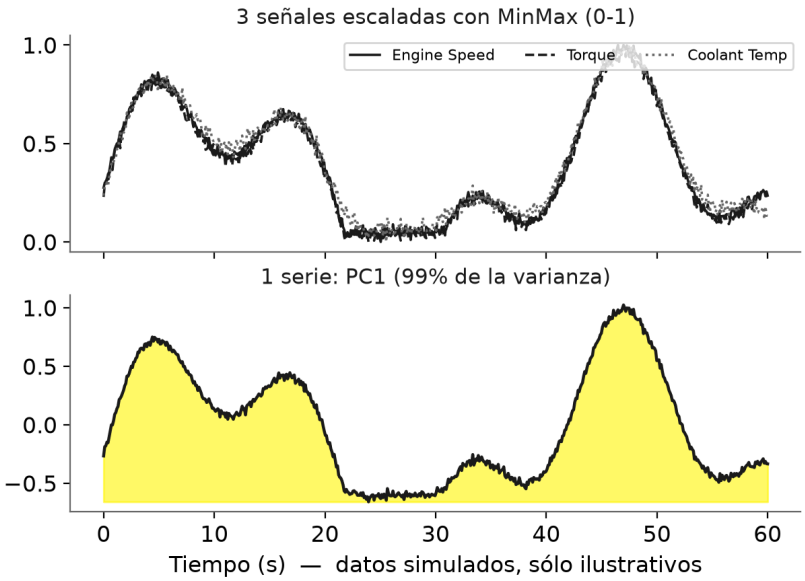
PCA es lineal

- Sólo encuentra rectas y planos
- Curvas y círculos se "revuelven" en la sombra

- **No lineal:** t-SNE y UMAP (visualización), Kernel PCA
- **Sólo variables numéricas continuas**: para categóricas existen MCA / FAMD
- Los componentes son menos **interpretables** que las variables originales; el signo es arbitrario

Ejemplo aplicado: motores Caterpillar

Señales del motor (3 por motor) → **MinMaxScaler (0 a 1)** → **PCA 1 componente** → **DTW-KMeans (series de tiempo)**



- Tesis del profesor: agrupar motores por su **comportamiento en el tiempo**
- Engine Speed (rpm), Torque y Coolant Temp tienen **unidades distintas** → MinMax
- Las 3 señales se mueven juntas → PCA las resume en **1 serie** por motor
- DTW compara series aunque estén **desfasadas**; K-Means agrupa con esa distancia

Código mínimo en scikit-learn

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA


X_scaled = StandardScaler().fit_transform(df[cols])


pca = PCA(n_components=0.95)    # o un entero
scores = pca.fit_transform(X_scaled)


pca.n_components_               # cuántos eligió
pca.explained_variance_ratio_   # varianza por PC
pca.components_                 # cargas
X_rec = pca.inverse_transform(scores)
```

Atributos clave

- **n_components**: entero o proporción de varianza
- **fit_transform**: ajusta y proyecta
- **components_**: una fila por PC
- **inverse_transform**: reconstruye
- PCA centra, pero **no escala**

Conclusiones

1. PCA busca las direcciones de máxima varianza: la sombra que conserva más información
2. Estandariza cuando las variables tienen unidades distintas
3. Elige componentes con varianza acumulada, codo y Kaiser según el objetivo
4. Interpreta cada componente con sus cargas y el biplot: es una mezcla, no una variable
5. Es lineal: para estructuras curvas usa t-SNE o UMAP